

PHYS30053 Assignment 1 - Option B

R. Durandet — [REDACTED]

School of Physics, University of Bristol.

Word count: 1098

(Dated: December 3, 2025)

1. METHODS

In this model, each event is statistically independent - meaning that this simulation was trivially parallelisable. Simultaneously calculating N events over multiple cores instead of one (native)[1] greatly reduces computation times for large N (Appendix, 6.6.1, Fig.7), allowing for larger datasets and thus increased statistical precision.

A single function was used to completely generate each individual event, with global parameters such as mean beam velocity or detector positions taken as arguments. The `ipyparallel` library was used for processing, due to its compatibility with Jupyter notebooks.

1.1. Random Variable Sampling

Seeds were used to ensure reproducibility across random sampling of velocity, lifetime, decay direction, and detector smearing.

The daughter particle direction was sampled uniformly over the full solid angle by drawing azimuthal and polar angles as:

$$\begin{aligned}\phi &\sim \text{Uniform}(0, 2\pi) \\ u &\sim [0, 1], \text{ such that:} \\ \theta &= \arccos(2u - 1) \sim [0, \pi].\end{aligned}$$

This method ensured uniformity over $\cos \theta$, which in turn ensured solid angle uniformity [2].

1.2. Detector Hits

Next, each daughter decay direction was used to calculate detector hits as explained in [3]. This was done within a loop iterating through detectors in order of increasing distance from the decay position.

To optimise this, conditions were set to allow the loop to break early. First, if a daughter direction has a negative z -component, it would never hit any detectors (as detectors work unidirectionally), so no calculation is necessary. Similarly, if a particle misses a detector, then geometrically it will miss all that remain. Given the low hit probabilities observed in this setup, these optimisations eliminated $\sim 87\%$ of irrelevant detector calculations (Appendix, 6.6.2).

1.3. Returning Results and Data Handling

This function returned results as a dictionary to facilitate conversion into a pandas DataFrame for analysis.

To optimise data transfer during parallel processing, the function only populated dictionary entries for smear/un-smear coordinates when a hit was recorded. As long as one result contains these columns, the pandas dataframe automatically fills missing columns with NaN values.

Since only $\sim 4.5\%$ of 10 million particles hit the first detector in this setup (see Fig.1), this conditional storage reduces the volume of data being serialised and transferred between processes, which is typically a key bottleneck. Avoiding unnecessary coordinate entries for the remaining 9.55 million no-hit particles prevents over 76 million coordinate pairs (2 arrays $\times$ 4 detectors) from being pickled and sent to worker processes [4, 5]. With float64 using 8 bytes per value, this corresponds to approximately 1.2 GB of reduced data transfer overhead.

2. RESULTS

2.1. Simulation

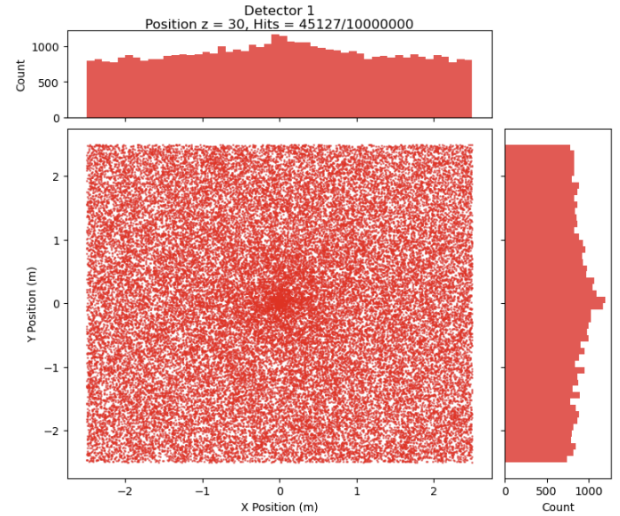


FIG. 1: Example scatter plot of smeared detector hits accompanied by x/y axis hit distribution histograms. Shows the front detector at $z = 30\text{m}$. Proportion of hits is $\sim 4.5\%$ of particle counts.

Fig.1 shows the spatial distribution of hits on the front detector, displaying uniform coverage across the detector face. The axis histograms show increased density toward the centre, reflecting the larger solid angle subtended by the central region which leads to increased hit probabilities.

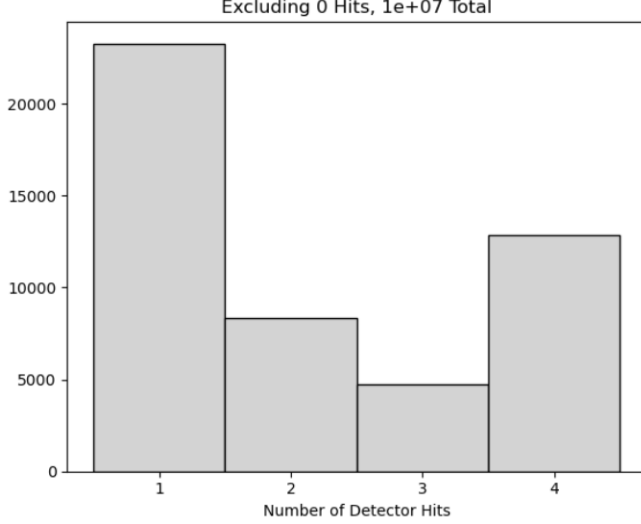


FIG. 2: Histogram of the number of detector hits per particle, excluding particles with 0 hits.

Fig.2 demonstrates the impact of the experiment geometry on detector hits. For a particle to hit only 2 or 3 detectors, it must scatter into narrow angular windows, making hitting just 1 or 4 detectors much more likely.

2.2. Reconstruction and Analysis

To carry out the experiment's goal of measuring the lifetime, the detector measurements needed to be reconstructed into decay positions and geometrically normalised so that a fit of the decay position PDF could be used to find τ .

Linear regression was used to fit each particle's trajectory across detectors and identify its intersection with the z-axis as the reconstructed decay vertex. Reconstructions were retained only if this vertex lay within 0.1 m radial distance of the z-axis. Only particles that hit all detectors were used for this, in order to minimise regression error.

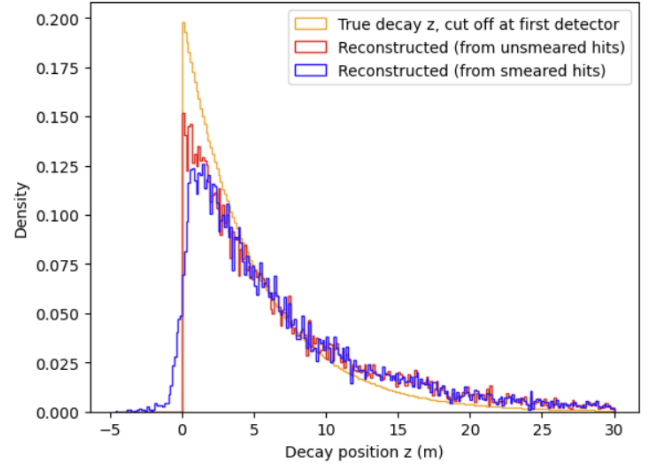


FIG. 3: Reconstructed particle decay position density histogram. 250 bins.

Fig.3 shows a positive bias between reconstructed data (both smeared/unsmeared) and true distributions. This is due to the geometrical bias inherent in measurements. Particles further away from the last detector are geometrically less likely to hit it than particles that decay nearer. Therefore, it is necessary to divide each bin by its respective geometric hit probability (derived in [6]).

For bin-by-bin error, regression and counting errors are combined using:

$$\sigma_{\text{bin}}^2 = \sigma_{\text{regression}}^2 + \sigma_{\text{counting}}^2$$

where $\sigma_{\text{counting}} = \sqrt{N_{\text{bin}}}$

and $\sigma_{\text{regression}}^2 = \sum_{i \in \text{bin}} \sigma_i^2$

so $\sigma_{\text{bin}}^2 = \sum_{i \in \text{bin}} \sigma_i^2 + N_{\text{bin}}$.

The smeared reconstruction curve in Fig.3 and its errors were thus geometrically normalized and plotted in Fig.4.

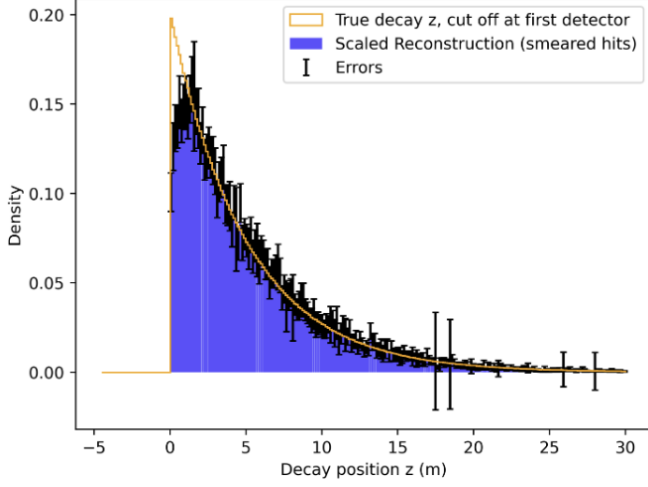


FIG. 4: Geometrically normalized measurements with errors compared against true distribution. 250 bins. All bins where $z < 0$ were dropped, due to it being unphysical. White lines are to plotting artifacts due to high bin count.

With the smeared results normalized, they could be fitted to the probability density function of decay position z . Assuming experimentalists know the velocity distribution's mean and standard deviation, the z probability density function (found by convoluting those of v and t [6]) was fitted to the measured data to find τ . This assumption is necessary, since simultaneously fitting μ , σ and τ results in degenerate solutions. The resulting integral over velocity was evaluated numerically using adaptive quadrature integration [7]. This was chosen for its efficiency with smoothly varying integrands and automatic error control, avoiding the need for manual step-size tuning required by fixed-step methods.

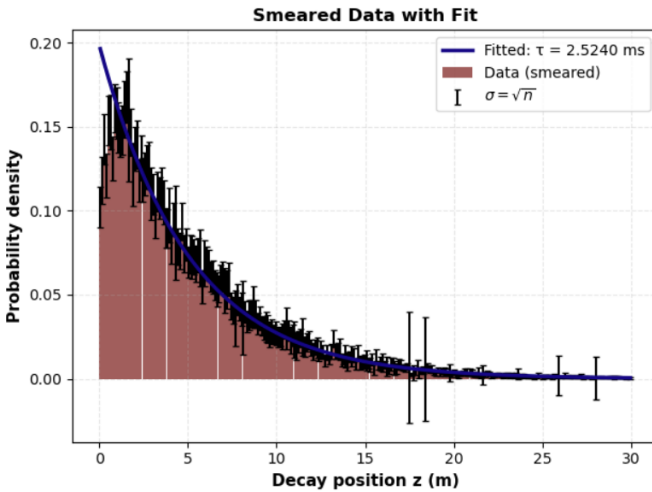


FIG. 5: Convolution of v and t (z PDF) fitted to measurements histogram - 250 bins.

White lines are to plotting artifacts due to high bin count.

The fit in Fig.5 yields a result for $\tau = 2.52 \pm 0.02\text{ms}$, with the result being 0.96 standard deviations from the true value

of $\tau = 2.5\text{ms}$ in this setup.

3. VALIDATION METHODOLOGY

The main validation strategy used involved fitting expected distributions to simulated results.

For example, to validate the distribution of decay positions, its PDF was analytically formulated from the convolution of the velocity and lifetime distributions [6] and fitted.

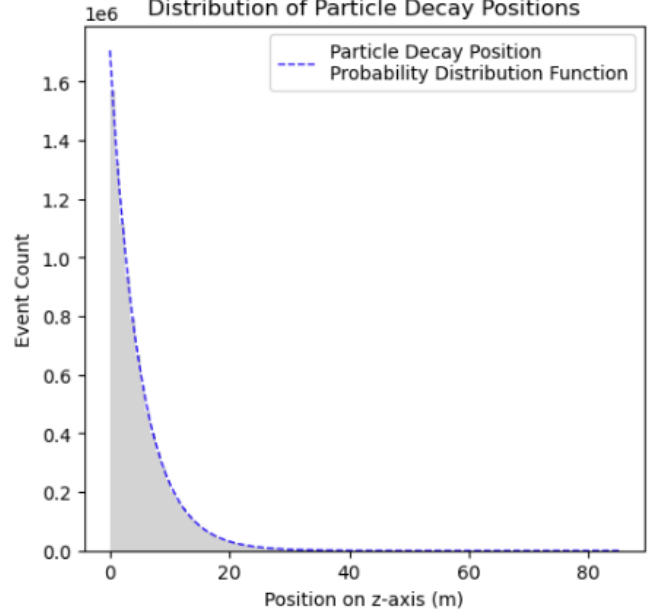


FIG. 6: Particle decay position histogram, PDF formulated from convolution of Gaussian velocity and Exponential lifetime distributions.

To validate the overall simulation, an independent analytical formulation and numerical evaluation of detector hit probabilities was performed and compared against simulation results [6]. This dual approach is essential for identifying potential systematic errors such as double counting of hits or incorrect geometric calculations.

Results agreed with analytic solutions within 0.289σ [6], validating the correctness of the hit-detection algorithm and geometric calculations.

4. CONCLUSIONS AND REFLECTIONS ON EXPERIMENTAL DESIGN

This project successfully simulated this particle physics experiment and reconstructed results to extract the mean particle lifetime. The measured value agreed to this input within 0.96 standard deviations.

All validation, both of the complete picture and on individual features, agreed with expectations as presented in [6].

Despite this success, several assumptions and limitations suggest avenues for improvement. Critically, the reconstruction step assumed knowledge of particle tracks. In this simplified setup, the low multiplicity (~ 0.01 hits/particle, Fig.2) made this assumption valid, but realistic detectors must solve the track-association problem at higher multiplicities. Simulating this combinatorial challenge [8] would be essential for a realistic detector model.

Furthermore, this assignment simplified treatment of detector smearing, leaving experiment results vulnerable. An airtight approach would use unfolding techniques to remove distortions introduced by detector errors [9].

Lastly, it was noticed that the quality of fitting was dependent on the number of bins n . As n increased, the geometric normalisation was sampled more finely and more points were available to constrain the fit. This however reduced the number of events per bin, causing the Poisson contribution to bin error $\sigma_{counting}^2 = N_{bin}$ to grow. Testing different n through trial and error confirmed this. Therefore, one could optimise n by defining a function describing the trade-off between statistical uncertainty and model resolution, and minimising it to maximise the expected precision of the lifetime fit.

5. USE OF GENERATIVE AI

Perplexity Research was used to generate boilerplate code. Examples include converting LaTeX formulas into python functions, generating integration code, and generating fitting code. When applicable, specified formulas, data inputs (with explanations), and desired output/formatting were provided. All code was reviewed and tested before application.

6. APPENDIX

6.1. Graph Testing Effect of Core Number on Computation Time

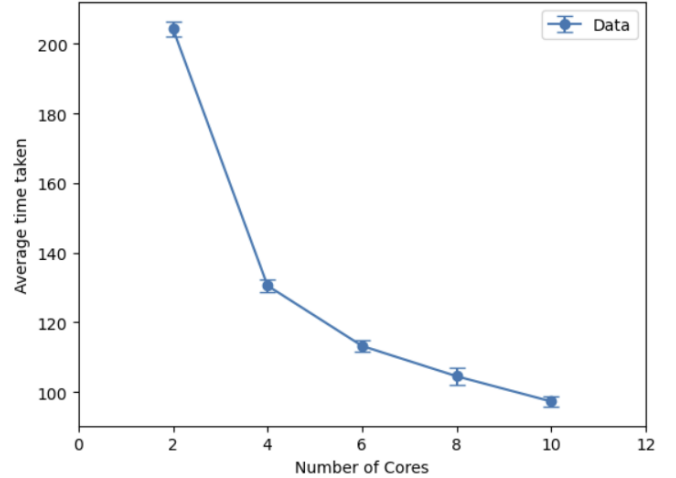


FIG. 7: Scatter graph testing computation time for different numbers of Cores used in processing 10 million particles. Each number of cores was tested 10 times, with standard deviation of times taken as error. The same 10 seed lists were used for all cores.

6.2. Detector calculation savings rate

To find how many detector calculations do occur, Fig.2 must be used.

Detector Calculations	Particle Number
1	5M (half of total)
2	46,000
3	26,000
4	17,500
sum	$\sim 5.1\text{M}$

TABLE I: Cumulative hits per detector using Fig.2 (approximative).

This amounts to ~ 5.1 million detector checks out of a potential 40 million detector checks if there were no such optimisations. This corresponds to a savings rate of 87.25%.

6.3. Remaining Detector Result Plots

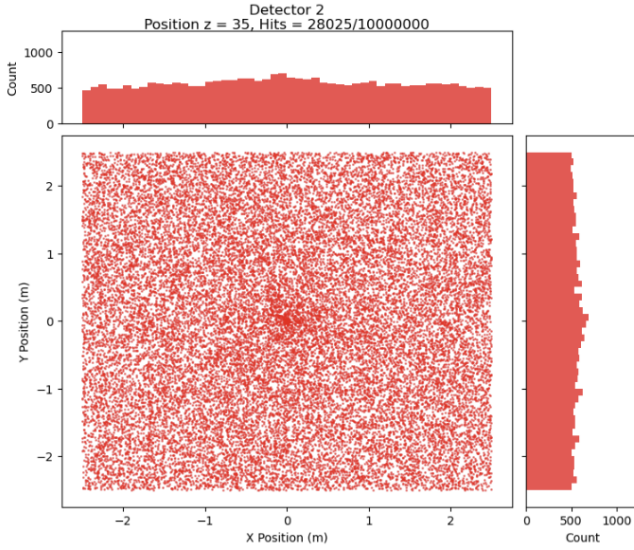


FIG. 8: Scatter plot of smeared detector hits accompanied by x/y axis hit distribution histograms.

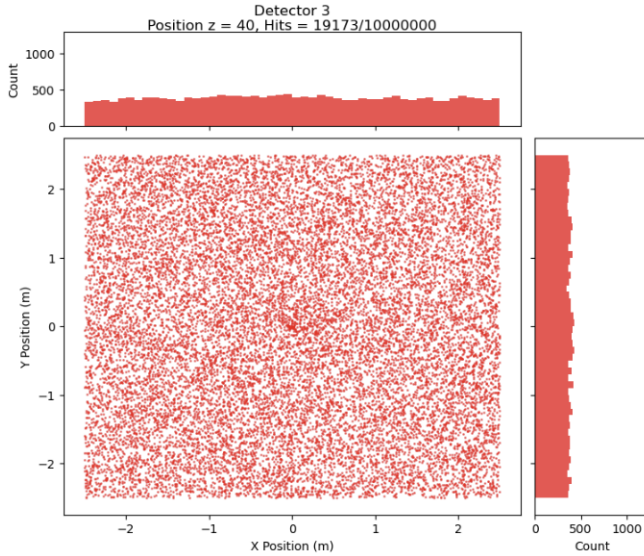


FIG. 9: Scatter plot of smeared detector hits accompanied by x/y axis hit distribution histograms.

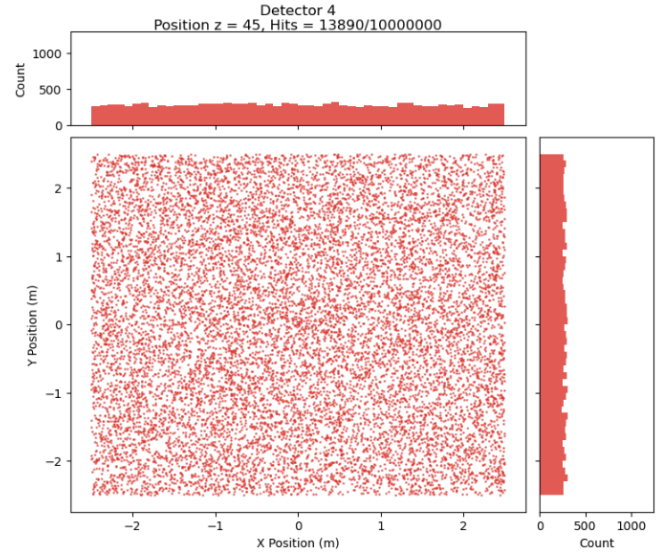


FIG. 10: Scatter plot of smeared detector hits accompanied by x/y axis hit distribution histograms.

7. REFERENCES

- [1] <https://peps.python.org/pep-0703/>. Pep 703 – making the global interpreter lock optional in cpython. *Python Enhancement Proposals*. URL <https://peps.python.org/pep-0703/>.
- [2] Weisstein, E. W. Sphere point picking. *MathWorld—A Wolfram Resource* URL <https://mathworld.wolfram.com/SpherePointPicking.html>. Accessed 29/11/2025.
- [3] University of Bristol, School of Physics. Assignment 1 - option b: Particle lifetime simulation. Assignment Brief (2025). PHYS30053: Advanced Computational Physics & Machine Learning.
- [4] Python Software Foundation. pickle — python object serialization (2024). URL <https://docs.python.org/3/library/pickle.html>. Python 3 Documentation.
- [5] IPython Development Team. Using dill to pickle anything (2024). URL <https://ipyparallel.readthedocs.io/en/latest/examples/Using%20Dill.html>. Ipyparallel Documentation.
- [6] Durandet, R. Phys30053 assignment 1b: Particle lifetime simulation (2025). Jupyter Notebook submission.
- [7] SciPy Development Team. scipy.integrate.quad (2024). URL <https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html>. SciPy Documentation.
- [8] ATLAS Collaboration. Software performance of the atlas track reconstruction for run 3 of the lhc (2023). URL <https://pure-oai.bham.ac.uk/ws/portalfiles/portal/223672259/s41781-023-00111-y.pdf>.
- [9] Schmitt, S. Data unfolding methods in high energy physics. *EPJ Web of Conferences* **137**, 11008 (2017). URL <http://dx.doi.org/10.1051/epjconf/201713711008>.